Лабораторная работа №3

QueueUp — автоматический подбор игроков. Node, npm, Bootstrap

1. Цель работы

Познакомиться с инструментами node и npm, научиться собирать простое веб-приложение на «чистом» JavaScript (без фреймворков) по компонентной архитектуре: разбиение на страницы (pages) и компоненты (components), работа с mock-данными (коллекцией) вместо реального бэкенда, подключение Bootstrap через npm, работа с DOM через getElementById / insertAdjacentHTML / addEventListener.

2. Задание и вариант

Задание: знакомство с node и npm; вёрстка интерфейса с карточками — страница списка с фильтрацией и страница «подробнее»; данные — через mock-объекты (коллекцию); кнопка добавления (копирует первую карточку); кнопка удаления карточки; в хедере на обеих страницах — кнопка «Домой».

Вариант: «Автоматический подбор игроков для игры. Услуги: карты, игровые локации, заявки: список карт участника в порядке его приоритета».

На основе текста варианта определены тема и обязательный компонент:

- Тема — подбор игроков для игры: сущности «заявка» (игрок, ищущий матч), с полями «игровая локация» и «карты»;
- Компонент — Список (Bootstrap list-group) с бейджами (badge) для номера приоритета, так как по условию внутри заявки нужно вывести карты участника именно в порядке его приоритета.

Визуальное оформление сайта сохранено в стилистике discord.com, как и в предыдущих лабораторных работах — тёмная палитра, фирменный цвет Blurple (#5865F2), фоновый градиент с цветным свечением.

3. Node.js, npm, package.json

- Node.js — платформа для выполнения JavaScript вне браузера, нужна здесь для запуска npm;
- npm — пакетный менеджер: команда npm install bootstrap скачивает библиотеку в node_modules;
- package.json — создан командой npm init -y, хранит название проекта и зависимости;
- package-lock.json — фиксирует точное дерево всех зависимостей для воспроизводимой сборки.

```
npm init -y
npm install bootstrap --save
```

4. Структура проекта

```
QueueUp/
├── node_modules/            (bootstrap — создаётся через npm install)
├── .gitignore
├── package.json
├── package-lock.json
├── index.html               — разметка + хедер с кнопкой "Домой"
├── main.js                  — точка входа, запуск MainPage, кнопка "Домой"
├── styles.css               — оформление в стиле discord.com
├── mocks/
│   └── applications.js      — mock-коллекция заявок (вместо бэкенда)
├── utils/
│   └── toast.js             — показ всплывающих уведомлений (Bootstrap Toast)
├── pages/
│   ├── main/index.js        — страница списка заявок + фильтры
│   └── application/index.js — страница одной заявки (подробнее)
└── components/
    ├── application-card/index.js  — карточка заявки (список, детали, удаление)
    ├── back-button/index.js       — кнопка "Назад"
    └── map-list/index.js          — список карт по приоритету (list-group + DnD)
```

5. Состав компонентов (для показа на защите)

Порядок показа: сначала объясняется состав компонентов проекта — какой файл за что отвечает:

Файл	Тип	Назначение

mocks/applications.js	mocks-коллекция	Массив заявок — имитация данных с бэкенда. Экспортируется напр

pages/main/index.js	страница	Список заявок: поиск по никнейму, фильтр по локации, кнопка «Добави

pages/application/index.js	страница	Карточка одной заявки из целикома: профиль игрока + список карт по

components/application-card/index.js	компонент	Карточка заявки: аватар, статус, кнопки «Подробнее», «Найти мат

components/map-list/index.js	компонент	Обязательный параметр — список карт (Bootstrap list-group) с

components/back-button/index.js	компонент	Кнопка «Назад к заявкам» на странице одной заявки.

utils/toast.js	утилита	Показывает всплывающее уведомление (BootstrapToast)
		использ

main.js	точка входа	Запускает MainPage при загрузке и привязывает к кнопку «Домой» в

6. Разбор кода

6.1. index.html и main.js — точка входа, хедер с кнопкой «Домой»

Хедер (с логотипом, навигацией и кнопкой «Домой») находится вне #root и поэтому не пересоздаётся при переключении между страницами — значит кнопка «Домой» автоматически присутствует и работает на обеих страницах, как и требуется по заданию:

```
<header class="site-header">

  ...

  <button id="home-button" class="nav-btn">🏠 Домой</button>

</header>

<div id="root"></div>
```

main.js привязывает обработчик клика на эту кнопку один раз при загрузке страницы:

```
function goHome() {

    const mainPage = new MainPage(root);

    mainPage.render();

}


document.getElementById('home-button').addEventListener('click', goHome);

goHome();
```

6.2. mocks/applications.js — mock-коллекция

Вместо запроса к серверу данные берутся из статического, но изменяемого JS-массива — экспортируется сам массив (а не функция getData()), чтобы кнопки «Добавить»/«Удалить» могли менять именно его, и изменения сохранялись при переходах между страницами в рамках одной сессии:

```
export const applicationsCollection = [

    { id: 1, nickname: 'ShadowFox', location: 'EU-West', maps: [...] },

    ...

]
```

6.3. MainPage — список, фильтрация, добавление, удаление

Фильтрация (по нику и по локации) выполняется на клиенте — на уже загруженной mock-коллекции, без повторных «запросов». Поля фильтров не пересоздаются при каждом вводе — перерисовывается только блок карточек (renderList), поэтому фокус в поле поиска не теряется:

```
getFilteredData() {

    return applicationsCollection.filter((item) => {

        const matchesSearch = item.nickname.toLowerCase()

            .includes(this.filters.search.trim().toLowerCase())

        const matchesLocation = this.filters.location === 'all'

            || item.location === this.filters.location

        return matchesSearch && matchesLocation

    })

}
```

Кнопка «Добавить заявку» копирует первый элемент коллекции, присваивает копии новый уникальный id и добавляет её в конец массива:

```
clickAdd() {

    const first = applicationsCollection[0]

    const newId = Math.max(...applicationsCollection.map((item) => item.id)) + 1

    const copy = { ...first, id: newId, nickname: `${first.nickname}
(копия)`, ... }

    applicationsCollection.push(copy)

    this.renderList()

}
```

Кнопка «✕» на карточке удаляет заявку из коллекции по id и тоже просто перерисовывает список:

```
clickDelete(appId) {

    const index = applicationsCollection.findIndex((item) => item.id === appId)

    if (index !== -1) applicationsCollection.splice(index, 1)

    this.renderList()

}
```

6.4. ApplicationCardComponent — карточка заявки

Принимает данные заявки и две функции-обработчика — переход на страницу «подробнее» и удаление. Каждая кнопка получает уникальный id по шаблону (open-app-${id}, delete-app-${id}), на который вешается addEventListener:

```
render(data, listener, deleteListener) {

    const html = this.getHTML(data)

    this.parent.insertAdjacentHTML('beforeend', html)

    this.addListeners(data, listener)

    this.addMatchFinderListener(data)

    this.addDeleteListener(data, deleteListener)

}
```

6.5. MapListComponent — обязательный компонент по варианту

Список карт участника в порядке приоритета на основе Bootstrap list-group — каждая карта получает круглый бейдж с порядковым номером:

```
getHTML(maps) {

    const itemsHTML = maps.map((map, index) => this.getItemHTML(map,
index)).join('')

    return `<ul class="list-group map-list">${itemsHTML}</ul>`

}
```

Список дополнительно интерактивен — карты можно перетаскивать мышью (нативный HTML5 Drag & Drop), после чего бейджи приоритета пересчитываются по новому порядку элементов в DOM (подробности — в разделе 7).

6.6. ApplicationPage — страница «подробнее»

Отрисовывает кнопку «Назад», профиль заявки и передаёт data.maps в MapListComponent:

```
render() {

    this.parent.innerHTML = ''

    const backButton = new BackButtonComponent(this.parent)

    backButton.render(this.clickBack.bind(this))

    ...

    const mapList = new MapListComponent(this.mapsRoot)

    mapList.render(this.data.maps)
```

```
}
```

7. Дополнительные фичи (сверх обязательного минимума)

7.1. Перетаскивание карт мышкой (Drag & Drop)

Любую карту в списке приоритета можно перетащить мышью на новую позицию — номера в бейджах пересчитываются сразу после отпускания. Реализовано на нативном HTML5 Drag & Drop API, без дополнительных библиотек:

- draggable="true" у каждого <li>;
- dragstart — запоминаем перетаскиваемый элемент, добавляем класс .dragging;
- dragover — вычисляем, после какого элемента сейчас курсор (по getBoundingClientRect), переставляем элемент в DOM;
- dragend — убираем .dragging, пересчитываем номера бейджей (updatePriorityBadges).

7.2. Кнопка «🎮 Найти матч» — имитация подбора

По клику кнопка блокируется, показывает спиннер и текст «Поиск...», статус заявки меняется на «В поиске»; через случайные 1.2–3 секунды статус становится «Матч найден», кнопка разблокируется, а в правом нижнем углу экрана всплывает уведомление (Bootstrap Toast) со случайным пингом — без перезагрузки страницы:

```
setTimeout(() => {

    const ping = Math.floor(18 + Math.random() * 55)

    statusBadge.textContent = 'Матч найден'

    showToast(`${data.nickname}: матч найден! Пинг ${ping} мс`)

}, searchTime)
```

8. Проверка работоспособности (автотесты через jsdom)

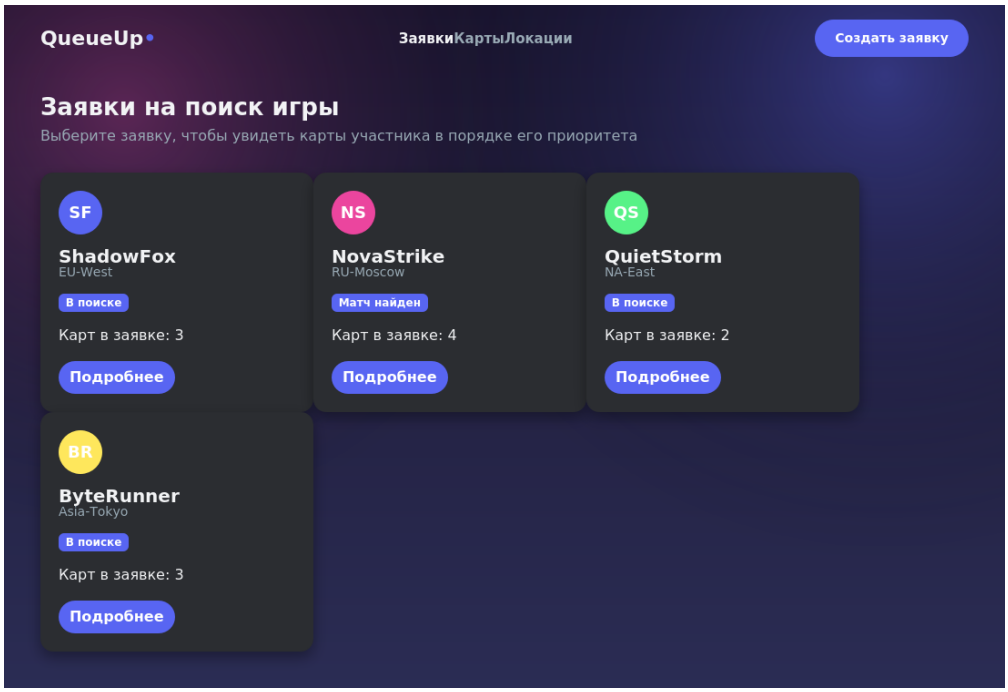
Логика проверена автоматическими тестами (эмуляция DOM в Node.js через jsdom — без реального браузера). Проверены: рендер и состав карточек, переход на страницу заявки, порядок и номера карт в list-group, работа фильтра по локации и поиска по нику, пустое состояние при отсутствии совпадений, добавление копии первой заявки, удаление заявки, кнопка «Домой» с обеих страниц, кнопка «Найти матч» (спиннер → результат → тост), пересчёт бейджей после перетаскивания карты. Все проверки — успешны:

```
Исходно карточек: 4                                    OK
```

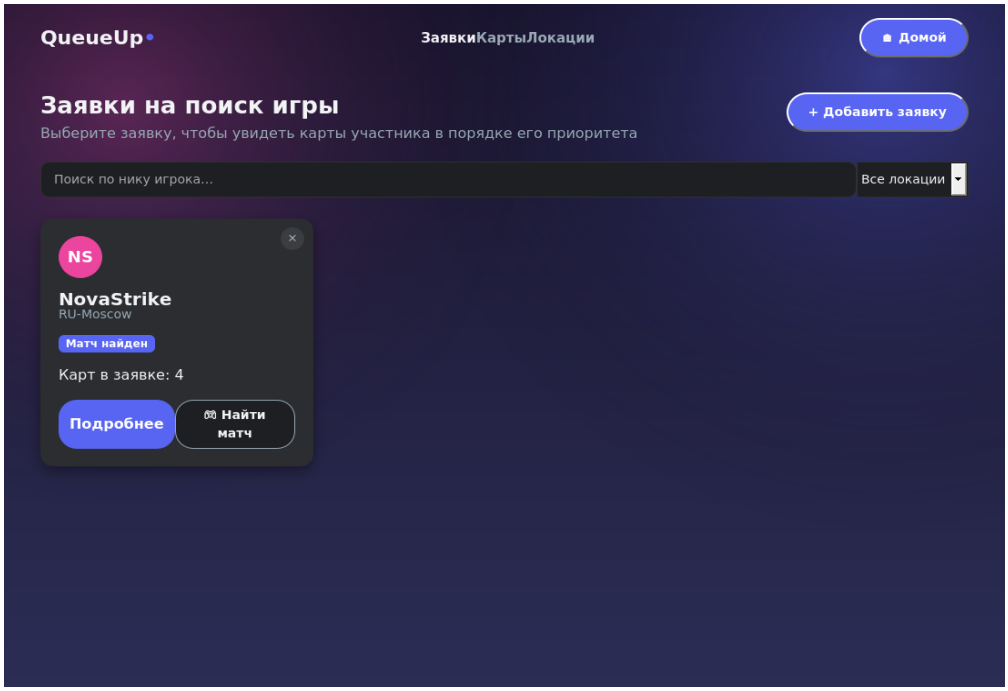
```
Опций в select (4 локации + "Все"): 5          OK
После фильтра по RU-Moscow карточек: 1         OK
Поиск "byte" находит карточек: 1               OK
Пустой результат карточек: 0, empty-state видим OK
Коллекция выросла на 1 после "Добавить"        OK
Новая заявка — копия первой, новый id          OK
Коллекция уменьшилась после "Удалить"          OK
Кнопка "Домой" работает со страницы заявки     OK
Спиннер/статус/тост фичи "Найти матч"          OK
Бейджи после перетаскивания карты: 1,2,3,4      OK
```

9. Результат — скриншоты

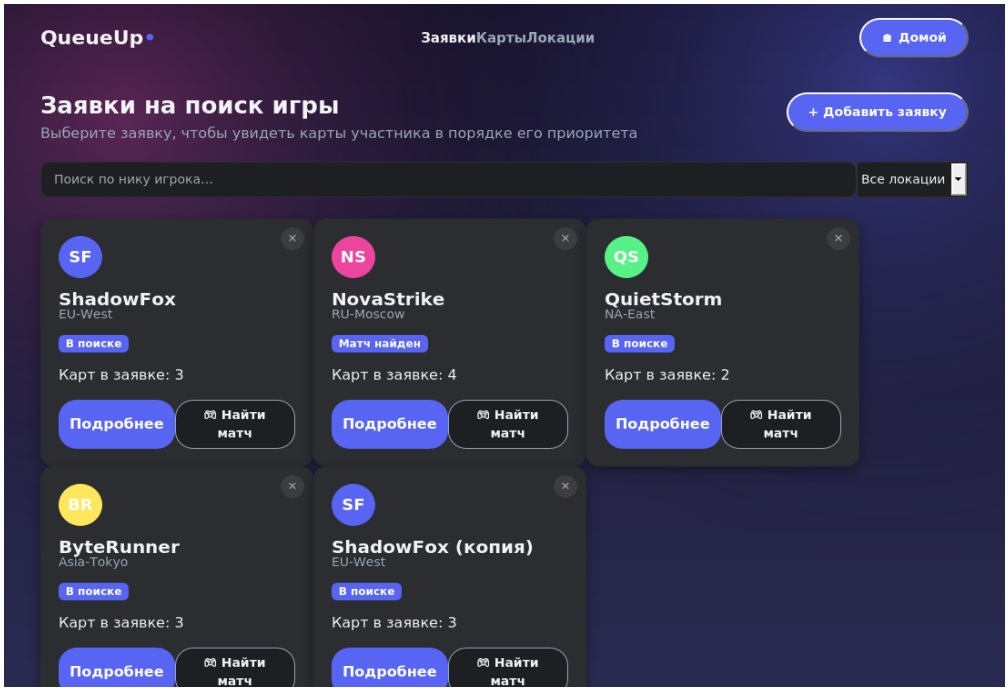
Главная страница — список заявок, панель фильтров, кнопка «Добавить заявку», кнопка «✕» на карточках, кнопка «Домой» в хедере:



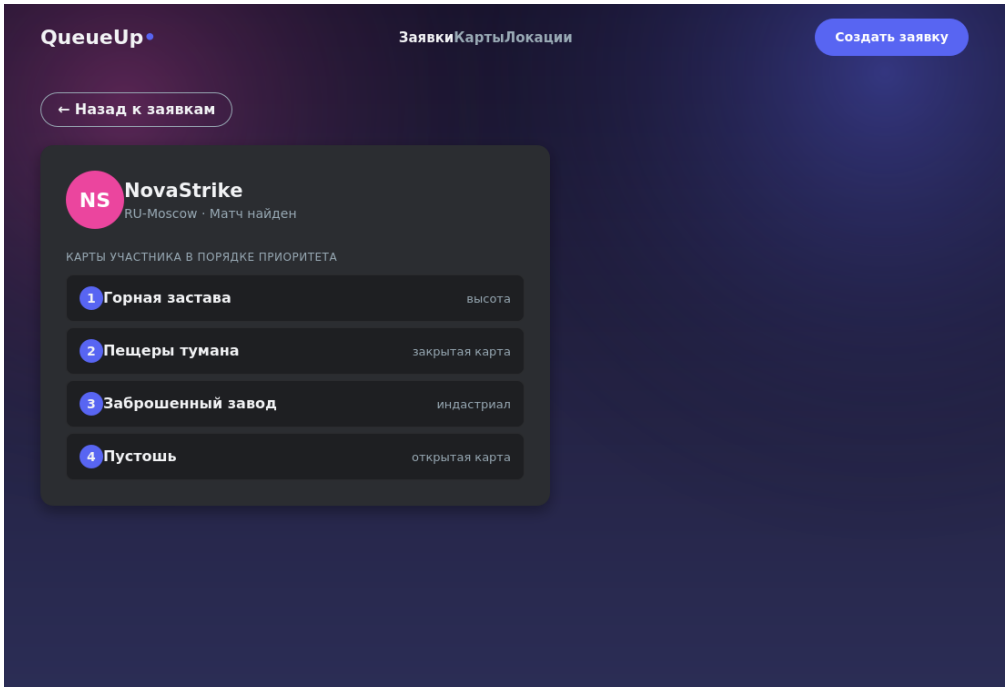
Результат фильтрации по локации RU-Moscow — остаётся одна подходящая заявка:



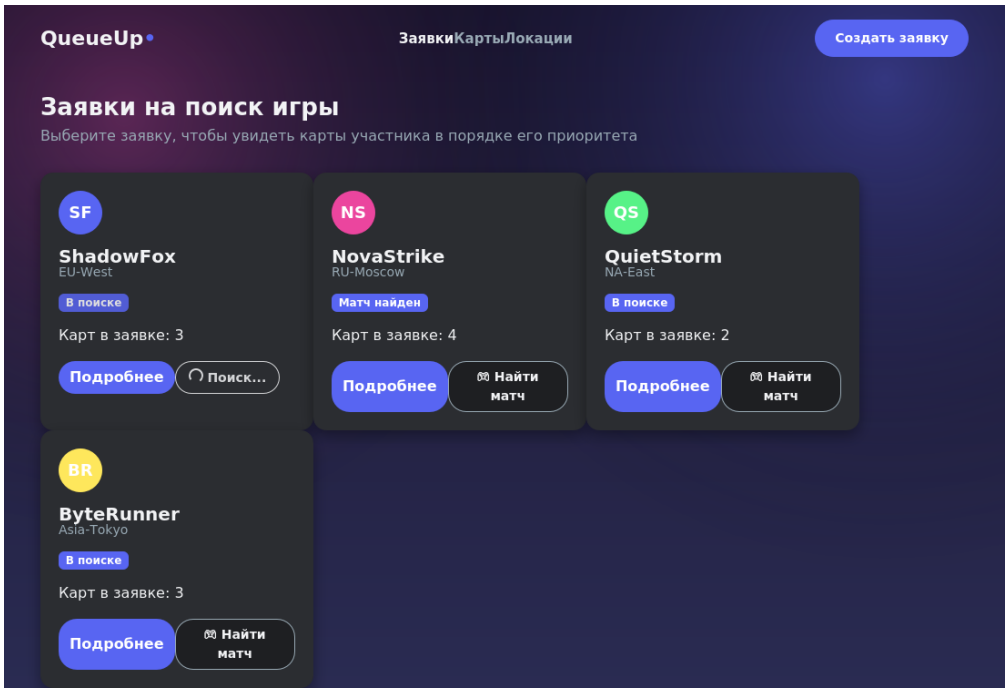
После нажатия «Добавить заявку» — внизу появилась копия ShadowFox:



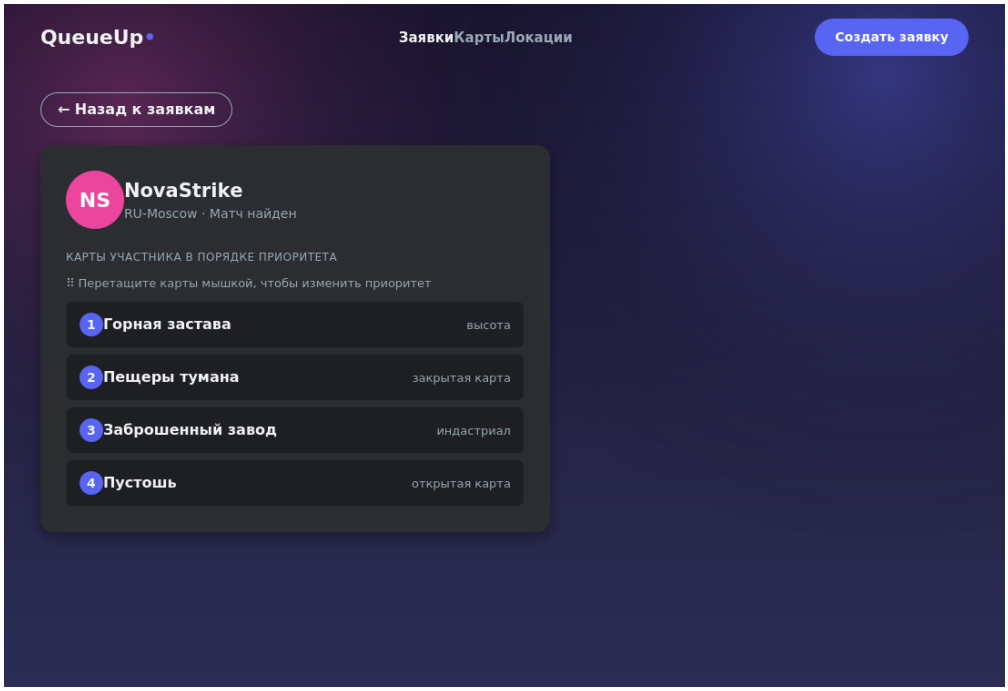
Страница заявки — профиль игрока и список карт в порядке приоритета (обязательный компонент варианта):



Фича «Найти матч» в процессе поиска (первая карточка):



Подсказка о перетаскивании над списком карт:



10. Демонстрация запроса страницы во вкладке Network

По заданию на защите нужно показать во вкладке Network загрузку HTML-страницы (GET-запрос) и содержимое Response. Порядок действий:

- Открыть сайт через Live Server в браузере (адрес вида http://127.0.0.1:5500/index.html);
- Открыть инструменты разработчика — клавиша F12 (или ПКМ на странице → «Просмотреть код»);
- Перейти на вкладку Network (Сеть);
- Обновить страницу — Ctrl+R (Cmd+R на Mac), чтобы запрос попал в список;
- В списке запросов найти самый первый — с именем index.html (или просто "/"), тип Type = document, метод Method = GET;
- Кликнуть по этому запросу — откроется панель с вкладками Headers / Preview / Response;
- Во вкладке Headers показать строку Request Method: GET и Status Code: 200;
- Во вкладке Response показать исходный HTML-код страницы, который вернул сервер (это тот же код, что лежит в index.html) — так наглядно видно, что браузер действительно запросил и получил HTML по сети (точнее, с локального сервера Live Server), а не просто открыл файл.

Это демонстрирует базовый принцип работы веб-страницы: браузер отправляет GET-запрос по адресу, сервер отвечает HTML-документом, после чего браузер его разбирает и достраивает остальное (CSS, JS-модули) отдельными запросами — их тоже можно увидеть в том же списке Network (style.css, main.js, bootstrap.min.css и т.д.).

11. Контрольные вопросы

1) Опишите структуру вашего проекта

Проект организован по компонентной архитектуре без фреймворков: точка входа main.js создаёт и рендерит первую страницу в корневой элемент #root. Код разделён на четыре зоны ответственности:

- pages/ — «экраны» приложения (MainPage, ApplicationPage), каждый — класс с конструктором (принимает родительский DOM-элемент), методом render() и обработчиками событий;
- components/ — переиспользуемые мелкие блоки интерфейса (карточка заявки, список карт, кнопка назад), из которых страницы собираются;
- mocks/ — заглушка данных (коллекция объектов) вместо реального бэкенда;
- utils/ — вспомогательные функции общего назначения (showToast), не привязанные к конкретной странице или компоненту.

Сам HTML-каркас (index.html) содержит только статичный хедер и пустой #root — всё содержимое страниц генерируется и вставляется в DOM через JavaScript (insertAdjacentHTML).

2) Что такое MVC?

MVC (Model-View-Controller) — архитектурный паттерн, разделяющий приложение на три слоя: Model хранит данные и бизнес-логику (в нашем проекте — mocks/applications.js и работа с этим массивом: фильтрация, добавление, удаление), View отвечает только за отображение (методы getHTML() в компонентах и страницах, которые строят разметку из данных), Controller связывает их — реагирует на действия пользователя и обновляет Model и View (обработчики кликов вроде clickAdd, clickDelete, clickCard в MainPage). Разделение упрощает поддержку кода: Model можно заменить на реальный запрос к серверу, не трогая View, а View можно переверстать, не трогая логику данных. В нашем проекте это разделение соблюдается нестрого (классы страниц совмещают роли View и Controller), но данные (Model) всё равно вынесены отдельно в mocks/.

3) Что такое роутинг?

Роутинг — это механизм, который определяет, какую «страницу» или компонент показать пользователю в зависимости от текущего адреса (URL) или внутреннего состояния приложения, и позволяет переключаться между ними без полной перезагрузки страницы с сервера. В полноценных SPA-роутерах (например, React Router) это обычно привязано к адресной строке через History API — у каждой страницы свой URL, работают кнопки «вперёд/назад» браузера, страницу можно открыть по прямой ссылке. В этом проекте реализован упрощённый («самодельный») роутинг без изменения URL: разные классы страниц (MainPage, ApplicationPage) просто поочерёдно рендерятся в один и тот же корневой элемент #root в зависимости от того, какая кнопка была нажата (clickCard → открыть заявку, clickBack/«Домой» → вернуться к списку). Адрес в браузере при этом не меняется.

4) Ключевые этапы рендера

Если говорить о рендеринге страницы браузером в целом, ключевые этапы такие: браузер получает HTML по GET-запросу и строит из него DOM-дерево; парсит подключённый CSS и строит CSSOM; объединяет DOM и CSSOM в render tree (только видимые элементы с их стилями); выполняет layout (reflow) — вычисляет точные размеры и позиции каждого элемента; затем paint —

закрашивает пиксели; и наконец compositing — собирает слои в итоговое изображение на экране.

Применительно к рендеру наших компонентов (метод render() в каждом классе) этапы такие: 1) получить/подготовить данные (из mocks-коллекции или входных параметров); 2) построить HTML-строку из данных через шаблонную функцию getHTML(); 3) вставить эту строку в DOM через insertAdjacentHTML; 4) навесить обработчики событий на только что созданные элементы (addEventListener) — на этом шаге элементы уже должны существовать в DOM, иначе document.getElementById их не найдёт.

12. Запуск проекта

- Распаковать архив проекта в отдельную папку;
- В терминале перейти в эту папку и выполнить npm install (установит зависимости из package.json — Bootstrap);
- Открыть папку в VS Code, запустить index.html через Live Server (ПКМ → Open with Live Server);
- Открывать index.html двойным кликом нельзя — используются ES-модули (type="module"), которые браузер по соображениям безопасности блокирует при загрузке страницы напрямую с диска (file:///); нужен локальный сервер, который и поднимает Live Server.